

01 — Data acquisition, non-IID partitioning, calibration split

Phase 1 of FedSafe-Fuse. Runs on Google Colab Free.

What this notebook produces

- `MyDrive/FedSafeFuse/partitions/medmnist_K3.json` — non-IID client partitions for MedMNIST OrganAMNIST
- `MyDrive/FedSafeFuse/partitions/brats_K3.json` — non-IID client partitions for BraTS 2020
- `MyDrive/FedSafeFuse/partitions/calibration_manifest.json` — 15% calibration hold-out indices
- Processed paired data persisted on Drive (`medmnist/*.npz`, `brats/processed/*.npz`)
- `MyDrive/FedSafeFuse/figures/phase1_sanity.png` — per-client distributions + sample slices

How to run

1. Open this notebook in Colab. **Runtime** → **Change runtime type** → **T4 GPU** is fine but not required for Phase 1.
2. Run cells **top to bottom**, in order.
3. You can stop after Section A (MedMNIST, ~10 min) and report back; Section B (BraTS, ~30–50 min) is heavier.
4. After completion, download the JSON manifests + sanity PNG to your local repo.

Outputs to download back

Drive path	Local path
<code>partitions/medmnist_K3.json</code>	<code>partitions/medmnist_K3.json</code>
<code>partitions/brats_K3.json</code>	<code>partitions/brats_K3.json</code>
<code>partitions/calibration_manifest.json</code>	<code>partitions/calibration_manifest.json</code>
<code>figures/phase1_sanity.png</code>	<code>figures/phase1_sanity.png</code>

Seed = 42 throughout.

```
# 0. Install dependencies and set global seeds
!pip install -q medmnist scikit-image opencv-python-headless einops n

import os, sys, json, random, time, gc, glob
import numpy as np
```

```

import torch
import matplotlib.pyplot as plt
import cv2

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

print("Python:", sys.version.split()[0])
print("Torch:", torch.__version__, "| CUDA available:", torch.cuda.is

```

 115.9/115.9 kB 5.0 MB/s et
 Python: 3.12.13
 Torch: 2.11.0+cu128 | CUDA available: True

```

# 1. Mount Google Drive for persistent storage
from google.colab import drive
drive.mount('/content/drive')

PROJECT_DRIVE = '/content/drive/MyDrive/FedSafeFuse'
for sub in ['medmnist', 'brats', 'brats/raw', 'brats/processed', 'par
    os.makedirs(f'{PROJECT_DRIVE}/{sub}', exist_ok=True)
print('Project root on Drive:', PROJECT_DRIVE)

```

Mounted at /content/drive
 Project root on Drive: /content/drive/MyDrive/FedSafeFuse

✓ Section A — MedMNIST OrganAMNIST (primary dataset)

Synthetic MRI/PET pairing via modality-specific transforms. Upsampled to 128×128.
 ~58k abdominal CT slices, 11 organ classes, non-IID via Dirichlet partitioning.

```

# 2. Download MedMNIST OrganAMNIST (train / val / test)
from medmnist import INFO, OrganAMNIST

info = INFO['organamnist']
NUM_CLASSES = len(info['label'])
print('Classes (11):', list(info['label'].values()))

train_ds = OrganAMNIST(split='train', download=True, root=f'{PROJECT_
val_ds   = OrganAMNIST(split='val',   download=True, root=f'{PROJECT_
test_ds  = OrganAMNIST(split='test',  download=True, root=f'{PROJECT_

# medmnist 3.x exposes raw arrays as .imgs / .labels
train_x = train_ds.imgs.astype(np.float32) / 255.0 # (N, 28, 28)
train_y = train_ds.labels.flatten().astype(np.int64)

```

```

val_x = val_ds.imgs.astype(np.float32) / 255.0
val_y = val_ds.labels.flatten().astype(np.int64)
test_x = test_ds.imgs.astype(np.float32) / 255.0
test_y = test_ds.labels.flatten().astype(np.int64)

print(f'Train: {train_x.shape} | Val: {val_x.shape} | Test: {test_x.s
print('Class counts (train):', dict(zip(*np.unique(train_y, return_co

```

```

Classes (11): ['bladder', 'femur-left', 'femur-right', 'heart', 'kidne
Train: (34561, 28, 28) | Val: (6491, 28, 28) | Test: (17778, 28, 28)
Class counts (train): {np.int64(0): np.int64(1956), np.int64(1): np.in

```

```

# 3. Synthetic MRI/PET pairing transforms (upsample 28 -> 128)
def upsample_to_128(arr_2d):
    return cv2.resize(arr_2d, (128, 128), interpolation=cv2.INTER_CUB

def simulate_mri_like(slice_128):
    """Stronger local contrast -> MRI-like appearance via CLAHE."""
    s = np.clip(slice_128, 0.0, 1.0)
    u8 = (s * 255).astype(np.uint8)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(u8).astype(np.float32) / 255.0
    return enhanced

def simulate_pet_like(slice_128):
    """Lower contrast + gamma + light blur -> PET-like metabolic sign
    s = np.clip(slice_128, 0.0, 1.0)
    windowed = np.power(s, 1.6)
    blurred = cv2.GaussianBlur(windowed, (5, 5), sigmaX=1.2)
    return np.clip(blurred, 0.0, 1.0).astype(np.float32)

def build_paired_split(x_raw, y_raw, name):
    out_path = f'{PROJECT_DRIVE}/medmnist/{name}_paired.npz'
    if os.path.exists(out_path):
        with np.load(out_path) as d:
            mri = d['mri']
            pet = d['pet']
            print(f'Loaded cached {out_path} shape={mri.shape}')
            return mri, pet
    n = x_raw.shape[0]
    mri = np.zeros((n, 128, 128), dtype=np.float32)
    pet = np.zeros((n, 128, 128), dtype=np.float32)
    for i in range(n):
        up = upsample_to_128(x_raw[i])
        mri[i] = simulate_mri_like(up)
        pet[i] = simulate_pet_like(up)
    np.savez_compressed(out_path, mri=mri, pet=pet, labels=y_raw)
    print(f'Saved {out_path} shape={mri.shape}')
    return mri, pet

t0 = time.time()
mri_train, pet_train = build_paired_split(train_x, train_y, 'train')
mri_val, pet_val = build_paired_split(val_x, val_y, 'val')

```

```
mri_test, pet_test = build_paired_split(test_x, test_y, 'test')
print(f'\nPairing took {time.time()-t0:.1f}s')
```

```
Loaded cached /content/drive/MyDrive/FedSafeFuse/medmnist/train_paired.n
Loaded cached /content/drive/MyDrive/FedSafeFuse/medmnist/val_paired.n
Loaded cached /content/drive/MyDrive/FedSafeFuse/medmnist/test_paired.n
```

```
Pairing took 78.2s
```

```
# 4. Non-IID partition (Dirichlet alpha=0.3) + 15% calibration hold-out
K = 3
ALPHA_DIRICHLET = 0.3
```

```
def dirichlet_partition(labels_pool_to_global_idx, labels, K, alpha, seed):
    """Partition global indices across K clients by Dirichlet over clients"""
```

```
    labels_pool_to_global_idx: list of absolute (global) indices from
    labels: full label array (indexed by global index)
    Returns: list of K lists of global indices.
    """
```

```
    rng = np.random.default_rng(seed)
    client_idx = [[] for _ in range(K)]
    pool_set = set(labels_pool_to_global_idx)
    for c in range(num_classes):
        idx_c = np.array([i for i in labels_pool_to_global_idx if labels[i] == c])
        rng.shuffle(idx_c)
        if len(idx_c) == 0:
            continue
        proportions = rng.dirichlet([alpha] * K)
        split_pts = (np.cumsum(proportions) * len(idx_c)).astype(int)
        chunks = np.split(idx_c, split_pts)
        for k in range(K):
            client_idx[k].extend(chunks[k].tolist())
    for k in range(K):
        rng.shuffle(client_idx[k])
    return [list(map(int, idxs)) for idxs in client_idx]
```

```
rng = np.random.default_rng(SEED)
n_train = train_x.shape[0]
perm = rng.permutation(n_train)
n_calib = int(0.15 * n_train)
calib_idx = perm[:n_calib].tolist()
train_pool_idx = perm[n_calib:].tolist()
```

```
client_idx_medmnist = dirichlet_partition(train_pool_idx, train_y, K, SEED)
```

```
medmnist_partition = {
    'dataset': 'medmnist_organamnist_axial',
    'seed': SEED,
    'num_clients': K,
    'dirichlet_alpha': ALPHA_DIRICHLET,
    'num_classes': NUM_CLASSES,
    'class_names': list(info['label'].values()),
```

```

'calibration_indices': [int(i) for i in calib_idx],
'client_indices': [[int(i) for i in client] for client in client_
'val_indices_global': list(range(int(val_x.shape[0]))),
'test_indices_global': list(range(int(test_x.shape[0]))),
'train_size': int(n_train),
'val_size': int(val_x.shape[0]),
'test_size': int(test_x.shape[0]),
'calibration_size': int(n_calib),
'per_client_class_counts': [
    [int((train_y[client] == c).sum()) for c in range(NUM_CLASSES
    for client in client_idx_medmnist
],
}

out = f'{PROJECT_DRIVE}/partitions/medmnist_K3.json'
with open(out, 'w') as f:
    json.dump(medmnist_partition, f, indent=2)
print(f'Saved {out}')
print('Per-client sizes:', [len(c) for c in client_idx_medmnist])
print('Calibration size:', len(calib_idx))
print('Per-client class counts:')
for k, counts in enumerate(medmnist_partition['per_client_class_count
    print(f' Client {k}: {counts}')
```

```

Saved /content/drive/MyDrive/FedSafeFuse/partitions/medmnist_K3.json
Per-client sizes: [7714, 15445, 6218]
Calibration size: 5184
Per-client class counts:
Client 0: [1115, 0, 7, 212, 21, 0, 1221, 2232, 2483, 258, 165]
Client 1: [450, 386, 1144, 8, 3355, 1406, 3078, 1109, 850, 835, 2824]
Client 2: [83, 778, 9, 1031, 3, 1829, 975, 15, 7, 1473, 15]
```

✓ Section B — BraTS 2020 (Kaggle redistribution)

Secondary dataset. T1/T2 axial slices, $\geq 1\%$ tumor mask, 128×128, normalised to [0,1].
Non-IID via HGG/LGG grade split across K=3 clients.

Idempotency

The next three cells (download, locate, extract) are **resumable** — if data is already on Drive from a previous run, they skip the heavy work. A clean fresh re-run finishes in a few minutes.

Getting your Kaggle API token (only needed if BraTS isn't already on Drive)

1. Sign in at <https://www.kaggle.com>
2. Go to <https://www.kaggle.com/settings> (or profile icon → **Settings**)
3. Scroll down to the **API** section → click **Create New Token**

4. Browser downloads `kaggle.json` (contains both username and key)

5. The next cell will prompt you to upload that file

```
# 5. Configure Kaggle API + download BraTS 2020 (skips if already on Drive)
BRATS_RAW_DIR = f'{PROJECT_DRIVE}/brats/raw'
os.makedirs(BRATS_RAW_DIR, exist_ok=True)
already = len(glob.glob(f'{BRATS_RAW_DIR}/**/*.BraTS20_Training_001', recursive=True))
print('BraTS already on Drive:', already)

if not already:
    # Need Kaggle credentials for download
    from google.colab import files
    print('Upload your kaggle.json (download via kaggle.com/settings)')
    uploaded = files.upload()

    # Accept any .json that contains both username and key
    import json as json_mod
    key_file = None
    for fname, content in uploaded.items():
        if fname.endswith('.json'):
            try:
                d = json_mod.loads(content)
                if 'username' in d and 'key' in d:
                    key_file = fname
                    break
            except Exception:
                pass
    assert key_file is not None, 'No valid Kaggle credentials file uploaded'

    import shutil
    os.makedirs('/root/.kaggle', exist_ok=True)
    shutil.move(key_file, '/root/.kaggle/kaggle.json')
    os.chmod('/root/.kaggle/kaggle.json', 0o600)
    print(f'Configured Kaggle credentials from {key_file}')

    !pip install -q kaggle
    print('Downloading BraTS 2020 from Kaggle (~4 GB, takes ~3 min)..')
    !kaggle datasets download -d awsaf49/brats20-dataset-training-validation
    print('Unzipping...')
    !cd {BRATS_RAW_DIR} && unzip -q -o brats20-dataset-training-validation.zip
    !rm -f {BRATS_RAW_DIR}/brats20-dataset-training-validation.zip
else:
    print('Skipping download - BraTS 2020 already extracted to Drive')

print('\nTop-level contents of BRATS_RAW_DIR:')
!ls {BRATS_RAW_DIR}
```

```
BraTS already on Drive: True
Skipping download - BraTS 2020 already extracted to Drive from a previous cell

Top-level contents of BRATS_RAW_DIR:
BraTS2020_TrainingData BraTS2020_ValidationData
```

```
# 6. Locate training dir + grade lookup (prefers 2020 subject-ID column)
case_anchor = glob.glob(f'{BRATS_RAW_DIR}/**/*.BraTS20_Training_001', recursive=True)
assert len(case_anchor) > 0, 'Could not find BraTS training cases. In
BRATS_TRAIN_DIR = os.path.dirname(case_anchor[0])
print('Training dir:', BRATS_TRAIN_DIR)

# Show actual file layout for one case (useful sanity check)
first_case = sorted(glob.glob(f'{BRATS_TRAIN_DIR}/BraTS20_Training_*'))[0]
print('\nExample files in', os.path.basename(first_case), ':')
for f in sorted(os.listdir(first_case)):
    print(' ', f)

import pandas as pd
mapping_candidates = glob.glob(f'{BRATS_RAW_DIR}/**/*.name_mapping.csv')
if mapping_candidates:
    mapping = pd.read_csv(mapping_candidates[0])
    print('\nname_mapping columns:', list(mapping.columns))
    # Prefer BraTS_2020 column, fall back to older years
    id_col = (next((c for c in mapping.columns if 'BraTS_2020' in c),
                  or next((c for c in mapping.columns if 'BraTS_2018' in c),
                          or next((c for c in mapping.columns if 'BraTS_2017' in c))))
    grade_col = next((c for c in mapping.columns if c.lower() == 'grade'), None)
    print(f'Using id_col={id_col}, grade_col={grade_col}')
    if id_col and grade_col:
        clean = mapping.dropna(subset=[id_col, grade_col])
        grade_lookup = dict(zip(clean[id_col], clean[grade_col]))
        print(f'Grade lookup has {len(grade_lookup)} entries')
        print('First 3:', list(grade_lookup.items())[:3])
    else:
        print('WARNING: could not auto-detect id/grade columns')
        grade_lookup = {}
else:
    print('WARNING: name_mapping.csv not found')
    grade_lookup = {}
```

Training dir: /content/drive/MyDrive/FedSafeFuse/brats/raw/BraTS2020_T

Example files in BraTS20_Training_001 :

```
BraTS20_Training_001_flair.nii
BraTS20_Training_001_seg.nii
BraTS20_Training_001_t1.nii
BraTS20_Training_001_t1ce.nii
BraTS20_Training_001_t2.nii
```

```
name_mapping columns: ['Grade', 'BraTS_2017_subject_ID', 'BraTS_2018_s
Using id_col=BraTS_2020_subject_ID, grade_col=Grade
Grade lookup has 369 entries
First 3: [('BraTS20_Training_001', 'HGG'), ('BraTS20_Training_002', 'H
```

```
# 7. Extract T1/T2 axial slices with >=1% tumor coverage, normalize to
import nibabel as nib
from collections import Counter
```

```

BRATS_OUT_DIR = f'{PROJECT_DRIVE}/brats/processed'

def slice_norm(x):
    """Percentile-based [0,1] normalisation, robust to outliers."""
    mn = np.percentile(x, 1)
    mx = np.percentile(x, 99)
    x = np.clip(x, mn, mx)
    return (x - mn) / (mx - mn + 1e-8)

def find_modality(case_dir, modality):
    """Find a .nii or .nii.gz file for a modality (t1, t2, seg). Excl
    for fname in os.listdir(case_dir):
        lower = fname.lower()
        if not (lower.endswith('.nii') or lower.endswith('.nii.gz')):
            continue
        stem = lower.rsplit('.nii', 1)[0]
        if modality == 'seg':
            if 'seg' in stem:
                return os.path.join(case_dir, fname)
        else:
            if stem.endswith('_' + modality):
                return os.path.join(case_dir, fname)
    return None

def extract_case(case_dir):
    case_id = os.path.basename(case_dir)
    t1_path = find_modality(case_dir, 't1')
    t2_path = find_modality(case_dir, 't2')
    seg_path = find_modality(case_dir, 'seg')
    if not (t1_path and t2_path and seg_path):
        return case_id, [], (t1_path, t2_path, seg_path)
    t1 = nib.load(t1_path).get_fdata().astype(np.float32)
    t2 = nib.load(t2_path).get_fdata().astype(np.float32)
    seg = nib.load(seg_path).get_fdata().astype(np.float32)
    H, W, S = t1.shape
    kept = []
    for s in range(S):
        seg_s = seg[..., s]
        if (seg_s > 0).mean() < 0.01:
            continue
        t1n = cv2.resize(slice_norm(t1[..., s]), (128, 128), interpol
        t2n = cv2.resize(slice_norm(t2[..., s]), (128, 128), interpol
        kept.append((s, t1n, t2n))
    return case_id, kept, None

case_dirs = sorted(glob.glob(f'{BRATS_TRAIN_DIR}/BraTS20_Training_*'))
print(f'Found {len(case_dirs)} case directories')

# Smoke test on first case to catch file-layout bugs before the long
cid0, slices0, miss0 = extract_case(case_dirs[0])
if not slices0:
    print(f'SMOKE TEST FAILED on {cid0}; missing paths: {miss0}')
    raise RuntimeError('Could not find expected files in first case.
print(f'Smoke test OK on {cid0}: kept {len(slices0)} slices')

```



```

all_meta, n_missing = [], 0
t0 = time.time()
for i, cdir in enumerate(case_dirs):
    case_id = os.path.basename(cdir)
    out_npz = f'{BRATS_OUT_DIR}/{case_id}.npz'
    if os.path.exists(out_npz):
        with np.load(out_npz) as d:
            n_slices = int(d['t1'].shape[0])
            all_meta.append({'case_id': case_id, 'grade': grade_lookup.get(case_id, 'LGG')})
            continue
    cid, kept, missing = extract_case(cdir)
    if not kept:
        n_missing += 1
        if n_missing <= 3:
            print(f' skip {cid} (missing: {missing})')
            continue
    idxs = np.array([s[0] for s in kept], dtype=np.int32)
    t1 = np.stack([s[1] for s in kept], axis=0)
    t2 = np.stack([s[2] for s in kept], axis=0)
    np.savez_compressed(out_npz, slice_idx=idxs, t1=t1, t2=t2)
    all_meta.append({'case_id': cid, 'grade': grade_lookup.get(cid, 'LGG')})
    if (i + 1) % 20 == 0:
        elapsed = time.time() - t0
        print(f' {i+1}/{len(case_dirs)} | {elapsed:.0f}s | kept {len(kept)} | skipped {n_missing}')

print(f'\nDone. Cases kept: {len(all_meta)} | skipped: {n_missing}')
print(f'Total slices: {sum(m["n_slices"] for m in all_meta)}')
print(f'Grade breakdown: {Counter(m["grade"] for m in all_meta)}')
print(f'Time: {time.time()-t0:.0f}s')

```

```

Found 369 case directories
Smoke test OK on BraTS20_Training_001: kept 70 slices
  skip BraTS20_Training_044 (missing: None)
  skip BraTS20_Training_063 (missing: None)
  skip BraTS20_Training_079 (missing: None)

Done. Cases kept: 365 | skipped: 4
Total slices: 17568
Grade breakdown: Counter({'HGG': 289, 'LGG': 76})
Time: 189s

```

```

# 8. Non-IID partition of BraTS by tumor grade (HGG / LGG) across K=3
cases_hgg = [m['case_id'] for m in all_meta if m['grade'] == 'HGG']
cases_lgg = [m['case_id'] for m in all_meta if m['grade'] == 'LGG']
rng = np.random.default_rng(SEED)
rng.shuffle(cases_hgg)
rng.shuffle(cases_lgg)
n_hgg, n_lgg = len(cases_hgg), len(cases_lgg)
print(f'HGG cases: {n_hgg} | LGG cases: {n_lgg}')

# 15% calibration hold-out drawn from ALL cases (HGG + LGG, proportionally)
all_cases_shuffled = list(cases_hgg) + list(cases_lgg)
rng.shuffle(all_cases_shuffled)
n_calib_cases = int(0.15 * len(all_cases_shuffled))

```

```

calib_cases = set(all_cases_shuffled[:n_calib_cases])

# Remove calib cases from the pools
pool_hgg = [c for c in cases_hgg if c not in calib_cases]
pool_lgg = [c for c in cases_lgg if c not in calib_cases]
n_h, n_l = len(pool_hgg), len(pool_lgg)

# Client allocation:
# Client 0: HGG-heavy (60% of pool_hgg, 0 LGG)
# Client 1: mixed (40% of pool_hgg, 20% of pool_lgg)
# Client 2: LGG-heavy (0 HGG, 80% of pool_lgg)
split_h0 = int(0.60 * n_h)
split_l1 = int(0.20 * n_l)
client_cases = [
    pool_hgg[:split_h0],
    pool_hgg[split_h0:] + pool_lgg[:split_l1],
    pool_lgg[split_l1:],
]

slice_lookup = {m['case_id']: m['n_slices'] for m in all_meta}
def make_slice_list(case_list):
    out = []
    for cid in case_list:
        for s in range(slice_lookup.get(cid, 0)):
            out.append([cid, int(s)])
    return out

client_slices = [make_slice_list(c) for c in client_cases]
calib_slices = make_slice_list(sorted(calib_cases))

grade_by_id = {m['case_id']: m['grade'] for m in all_meta}
brats_partition = {
    'dataset': 'brats2020_kaggle_redistribution',
    'seed': SEED,
    'num_clients': K,
    'calibration_cases': sorted(list(calib_cases)),
    'calibration_slices': calib_slices,
    'client_cases': client_cases,
    'client_slices': client_slices,
    'per_client_grade_counts': [
        {'HGG': sum(1 for c in lst if grade_by_id.get(c) == 'HGG'),
         'LGG': sum(1 for c in lst if grade_by_id.get(c) == 'LGG')}
        for lst in client_cases
    ],
    'per_client_slice_counts': [
        sum(slice_lookup.get(c, 0) for c in lst) for lst in client_ca
    ],
    'all_meta': all_meta,
}

out = f'{PROJECT_DRIVE}/partitions/brats_K3.json'
with open(out, 'w') as f:
    json.dump(brats_partition, f, indent=2)
print(f'Saved {out}')
for k in range(K):

```

```
print(f' Client {k}: {len(client_cases[k])} cases | {brats_parti
print(f' Calibration: {len(calib_cases)} cases | {len(calib_slices)}
```

HGG cases: 289 | LGG cases: 76

Saved /content/drive/MyDrive/FedSafeFuse/partitions/brats_K3.json

Client 0: 146 cases | 7055 slices | grade={'HGG': 146, 'LGG': 0}

Client 1: 111 cases | 5126 slices | grade={'HGG': 98, 'LGG': 13}

Client 2: 54 cases | 2633 slices | grade={'HGG': 0, 'LGG': 54}

Calibration: 54 cases | 2754 slices

✓ Section C — Unified calibration manifest + sanity figure

```
# 9. Combined calibration manifest
calibration_manifest = {
    'medmnist': {
        'calibration_indices': medmnist_partition['calibration_indice
        'size': len(medmnist_partition['calibration_indices']),
        'source': 'train_split_random_15pct_then_dirichlet',
    },
    'brats': {
        'calibration_cases': brats_partition['calibration_cases'],
        'calibration_slices': brats_partition['calibration_slices'],
        'size_cases': len(brats_partition['calibration_cases']),
        'size_slices': len(brats_partition['calibration_slices']),
        'source': 'random_15pct_case_holdout',
    },
}
out = f'{PROJECT_DRIVE}/partitions/calibration_manifest.json'
with open(out, 'w') as f:
    json.dump(calibration_manifest, f, indent=2)
print(f'Saved {out}')
```

Saved /content/drive/MyDrive/FedSafeFuse/partitions/calibration_manife

```
# 10. Sanity figure: per-client distributions + sample paired slices
fig, axes = plt.subplots(2, 4, figsize=(16, 7))

# Row 1: MedMNIST per-client class histograms + sample paired slices
for k in range(K):
    counts = medmnist_partition['per_client_class_counts'][k]
    axes[0, k].bar(range(NUM_CLASSES), counts, color='steelblue')
    axes[0, k].set_title(f'MedMNIST Client {k} (n={sum(counts)})')
    axes[0, k].set_xlabel('Organ class')
    axes[0, k].set_ylabel('Count')

    axes[0, 3].imshow(np.hstack([mri_train[0], pet_train[0]]), cmap='gray')
    axes[0, 3].set_title('MedMNIST: simulated MRI | PET')
    axes[0, 3].axis('off')

# Row 2: BraTS per-client grade bars + sample T1/T2 slice
```

```
for k in range(K):
    g = brats_partition['per_client_grade_counts'][k]
    axes[1, k].bar(['HGG', 'LGG'], [g['HGG'], g['LGG']], color=['crim
    axes[1, k].set_title(f'BraTS Client {k}\n{g["HGG"]} HGG + {g["LGG"]

if all_meta:
    sample_case = all_meta[0]['case_id']
    with np.load(f'{BRATS_OUT_DIR}/{sample_case}.npz') as d:
        s0 = d['t1'].shape[0] // 2
        t1_s = d['t1'][s0]
        t2_s = d['t2'][s0]
        axes[1, 3].imshow(np.hstack([t1_s, t2_s]), cmap='gray', vmin=0, vi
        axes[1, 3].set_title(f'BraTS {sample_case}: T1 | T2 (mid-slice)')
        axes[1, 3].axis('off')

plt.tight_layout()
fig_path = f'{PROJECT_DRIVE}/figures/phase1_sanity.png'
plt.savefig(fig_path, dpi=140, bbox_inches='tight')
plt.show()
print(f'Saved {fig_path}')
```



Place them under your local (partitions/) and (figures/) folders.

The heavy (.npz) files (MedMNIST paired arrays + per-case BraTS slices) stay on Drive — Phase 2 and Phase 3 notebooks will read from there directly.

After running, paste back to chat:

1. The printed output from the partition cells (per-client sizes, grade counts, total slices).
2. Confirmation that (phase1_sanity.png) looks reasonable.
3. Any errors or warnings.